Intelligent Pet Care Robot

Context-aware robot system using LLM, CV, and ROS2

Team컴공 | 2271107 이경선 | 2376087 류다현

https://github.com/intelligent-robotics-teamcs/robotics-project

Project Goal and Topic

Convert situation-based language into executable robot actions

- Understand user intent from natural language
- Detect objects with YOLOv8
- Generate an action sequence with an LLM planner
- Move TurtleBot3 using odom-based control
- Execute observe, wait, feed and report actions

Project Goal and Topic

Convert situation-based language into executable robot actions

Example Request:

“He seems hungry…”



“You are a ROS2 robot action planner for a pet-care robot.”

Output Action Sequence:

```
{object1 : dog},
{object2: apple},
 {action : feed}
```

Example Scenario

Intent, perception, planning, and robot execution in one flow

1. Request

“He seems hungry…”

2. Plan

apple → pet → feed

3. Move

approach target zone

4. Report

Report publish status

Example Scenario

Intent, perception, planning, and robot execution in one flow

1. Request

“He seems hungry...”

2. Plan

apple → pet → feed

Key idea: the robot executes a sequence, not just a single command.

3. Move

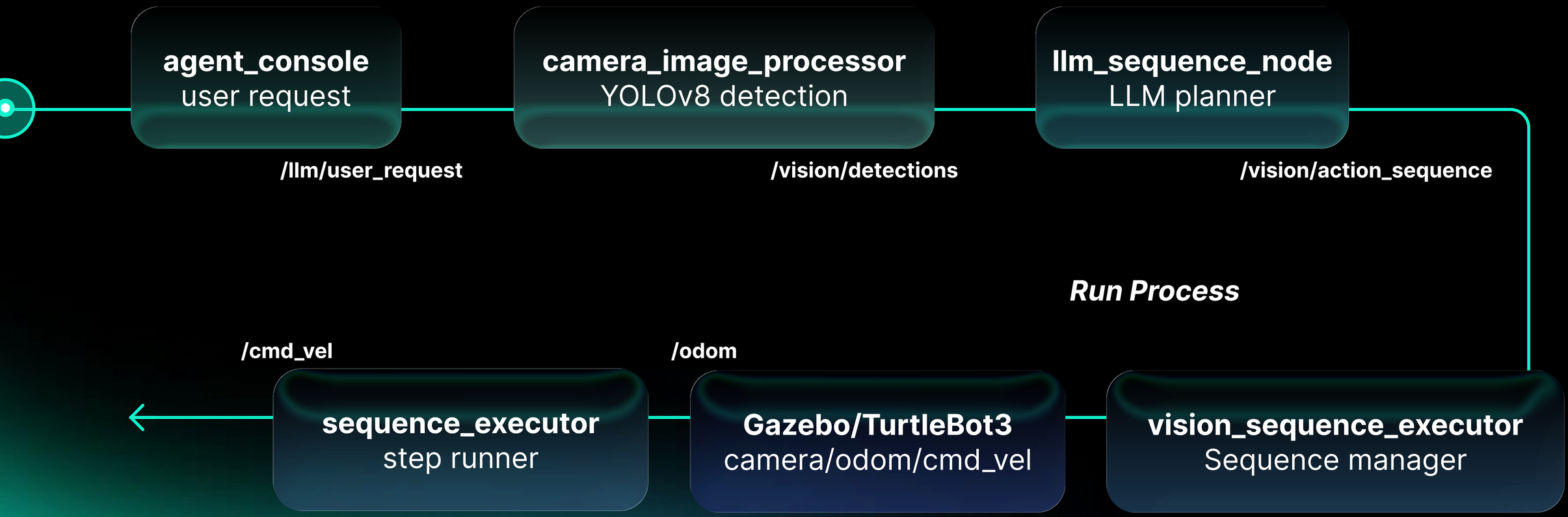
approach target zone

4. Report

Report publish status

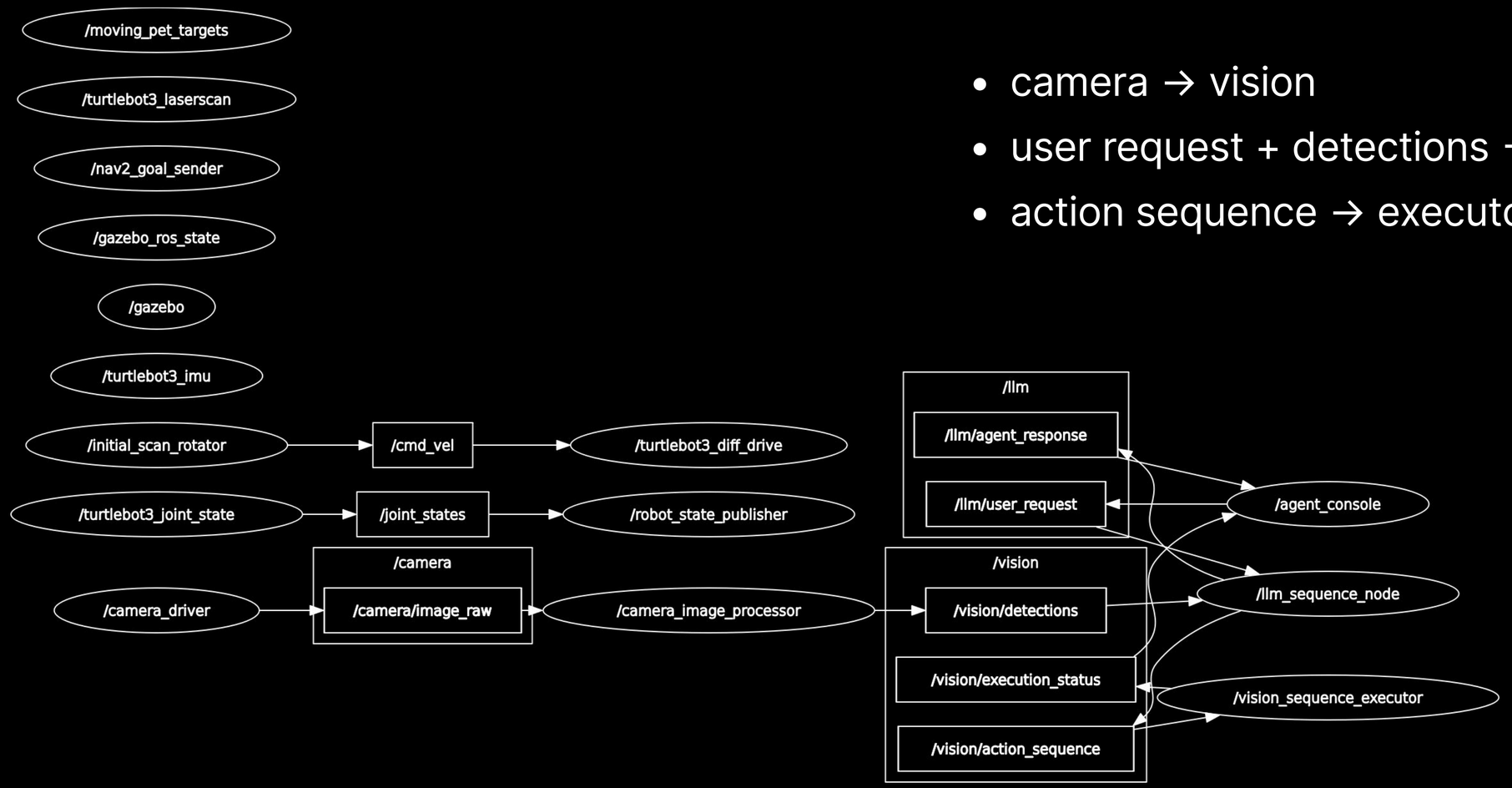
System Architecture

ROS node composition and topic connections



System Architecture

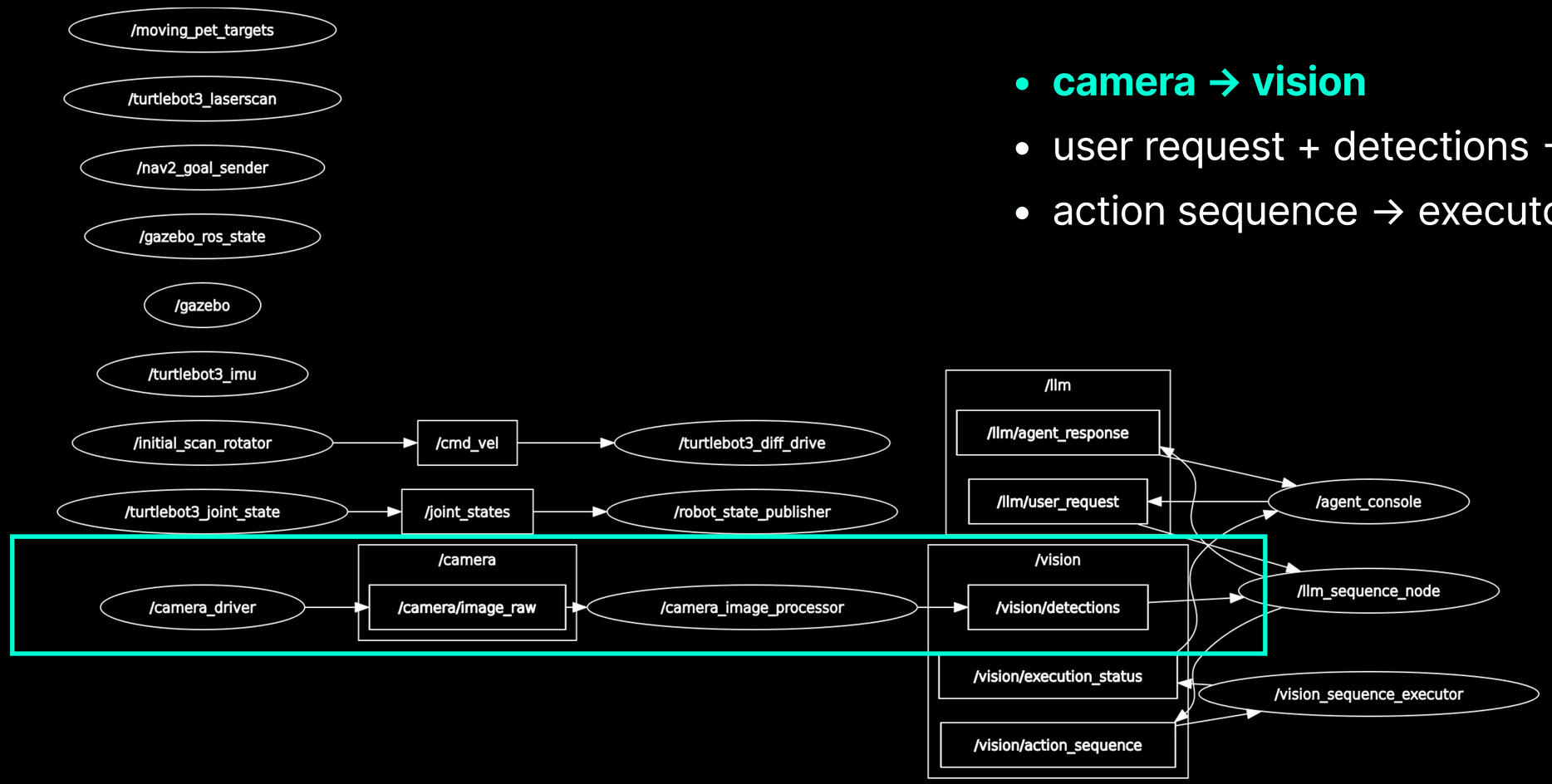
ROS node composition and topic connections



- camera → vision
- user request + detections → LLM
- action sequence → executor → sequence

System Architecture

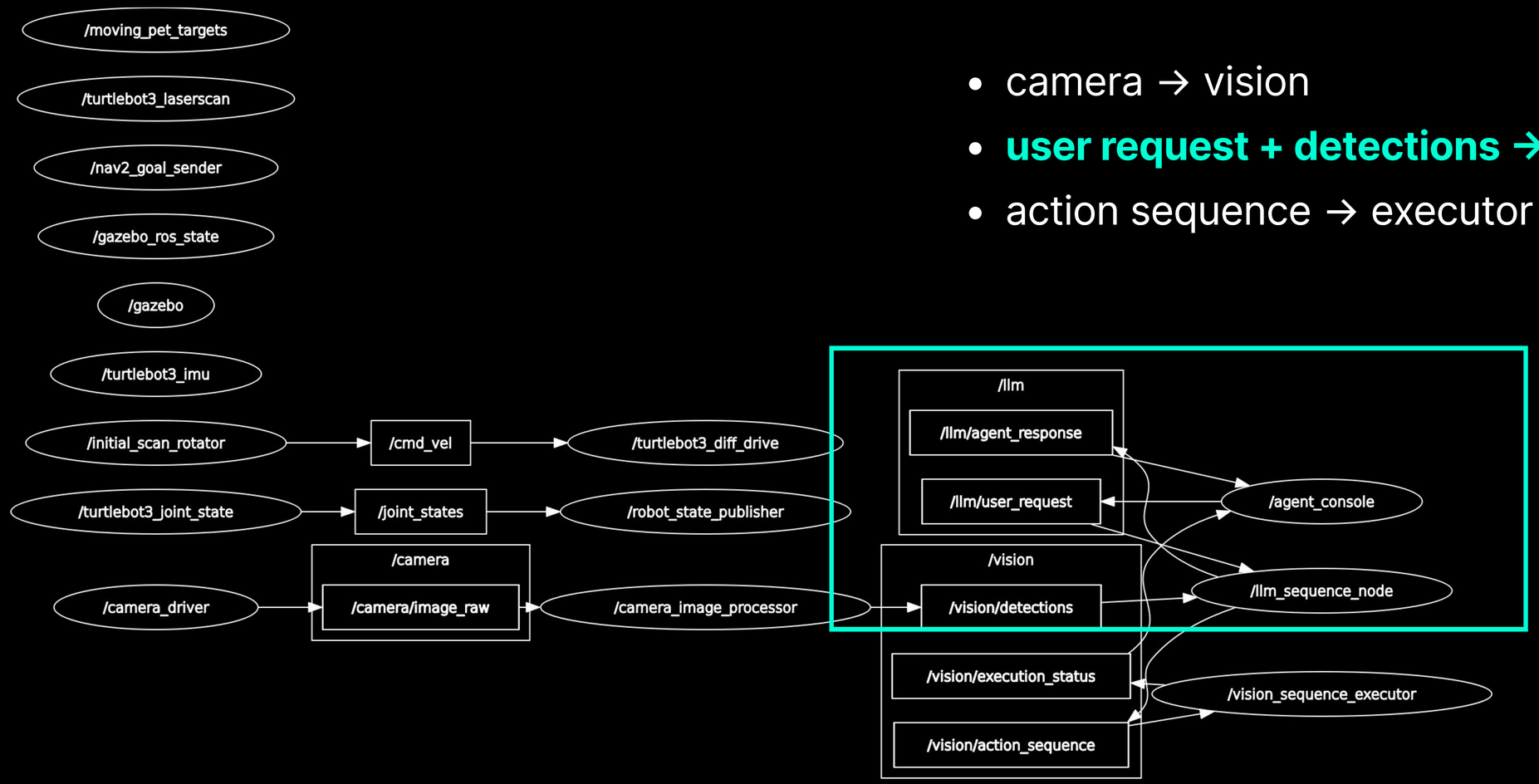
ROS node composition and topic connections



- **camera → vision**
- user request + detections → LLM
- action sequence → executor → sequence

System Architecture

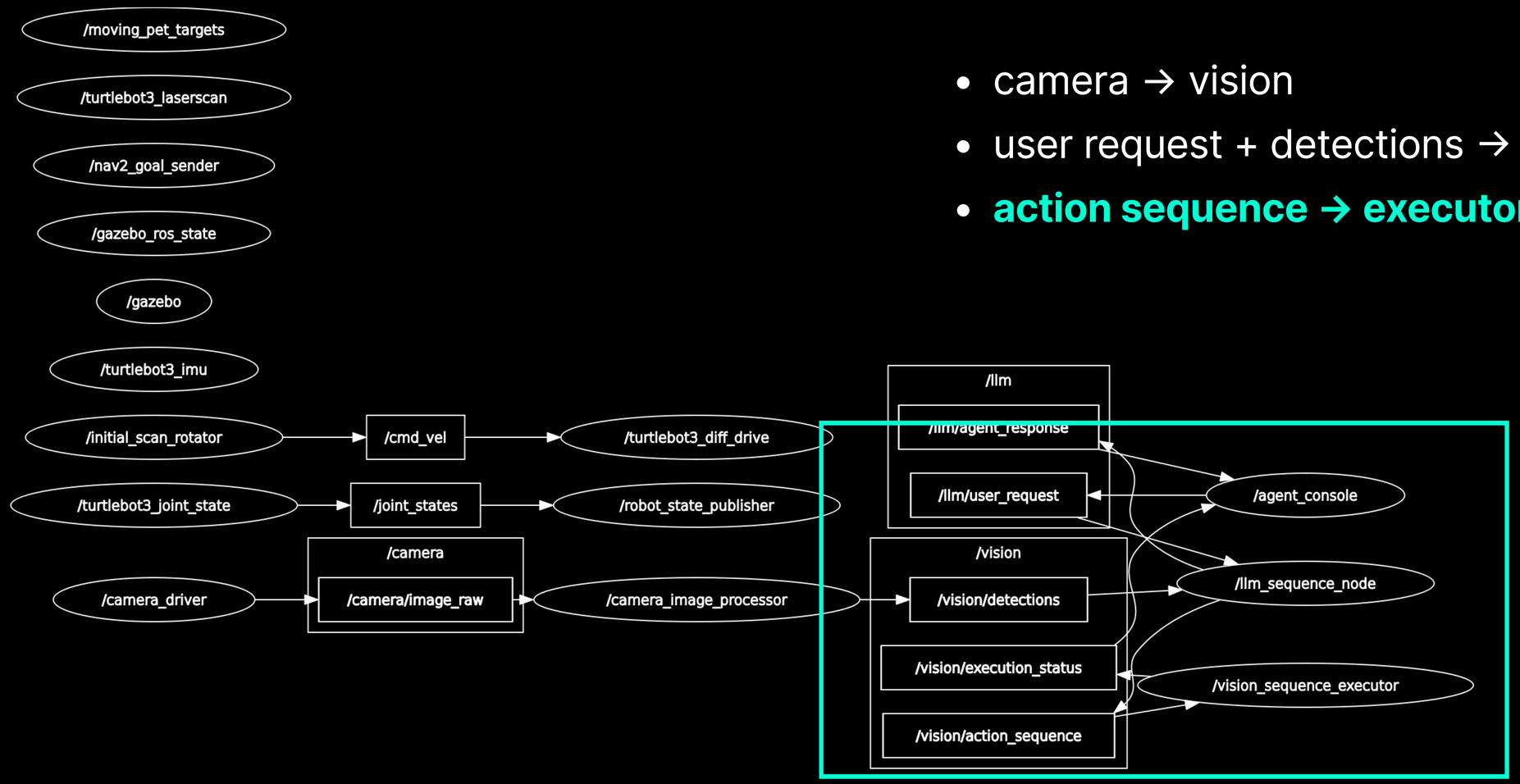
ROS node composition and topic connections



- camera → vision
- **user request + detections → LLM**
- action sequence → executor → sequence

System Architecture

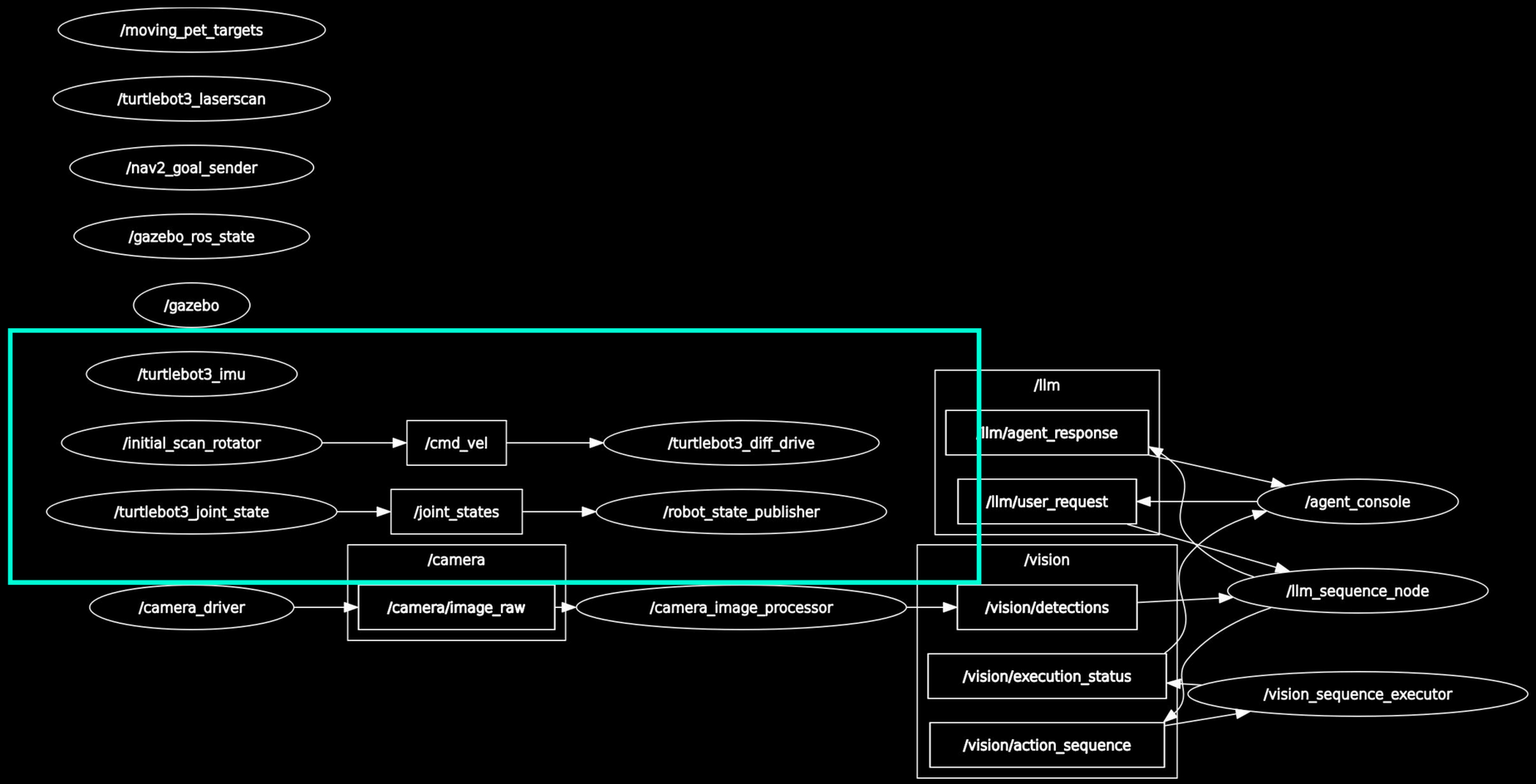
ROS node composition and topic connections



- camera → vision
- user request + detections → LLM
- **action sequence → executor → sequence**

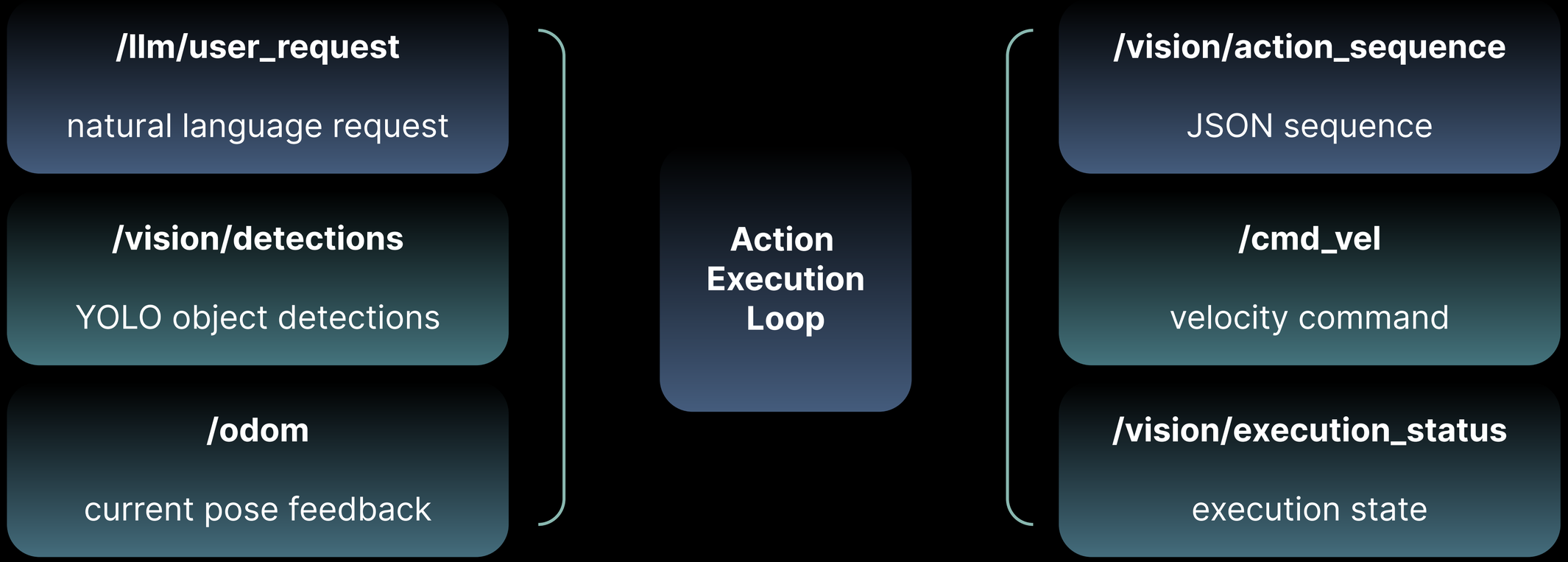
System Architecture

ROS node composition and topic connections



Data Flow

User request, vision detections, and odom feedback merge into one execution loop



Implementation: LLM Planner

Constrain LLM output to predefined JSON actions

- Uses object and action schemas
- Supports OpenAI API
- Uses deterministic fallback without API key
- Adds JSON parsing, validation, and fallback logic

```
Required JSON shape:
{{
  "comment": "short Korean comment",
  "sequence": [
    {{
      "step_id": 1,
      "action": "approach",
      "object": "apple",
      "params": {{
        "timeout_sec": 60.0,
        "goal_tolerance_m": 0.25,
        "retry_count": 2
      }}
    }},
    {{
      "step_id": 2,
      "action": "report",
      "object": null,
      "params": {{
        "message": "sequence completed"
      }}
    }}
  ]
}}
"""
```

Implementation: Vision Pipeline

Constrain LLM output to predefined JSON actions

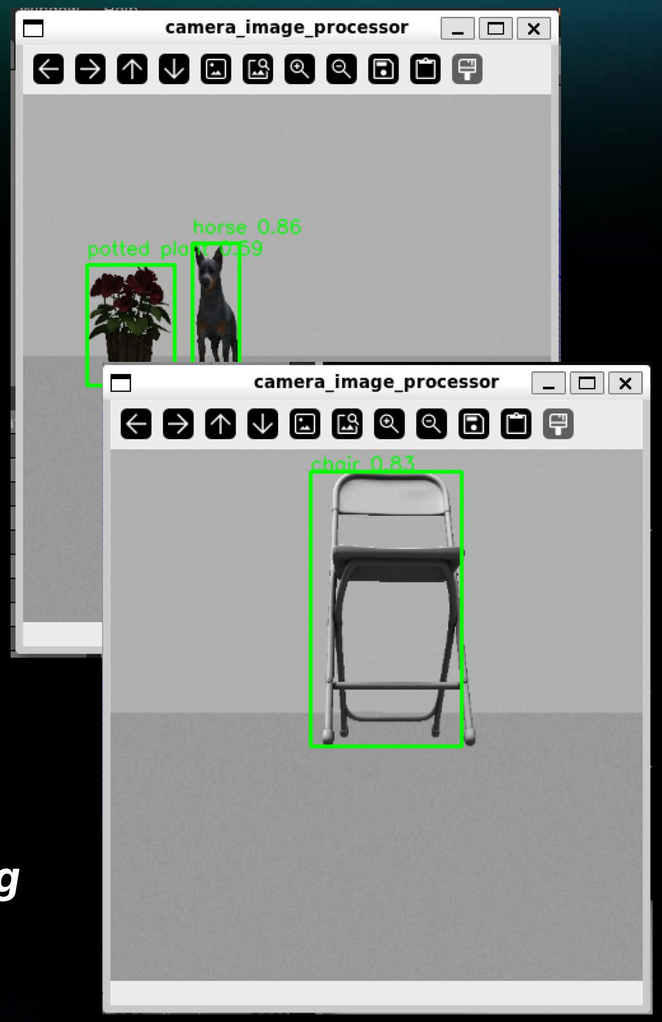
Camera Topic
/camera/image_raw

cv_bridge
ROS Image → OpenCV

YOLOv8
label & confidence & bbox

Filter
object schema

Publish
/vision/detections

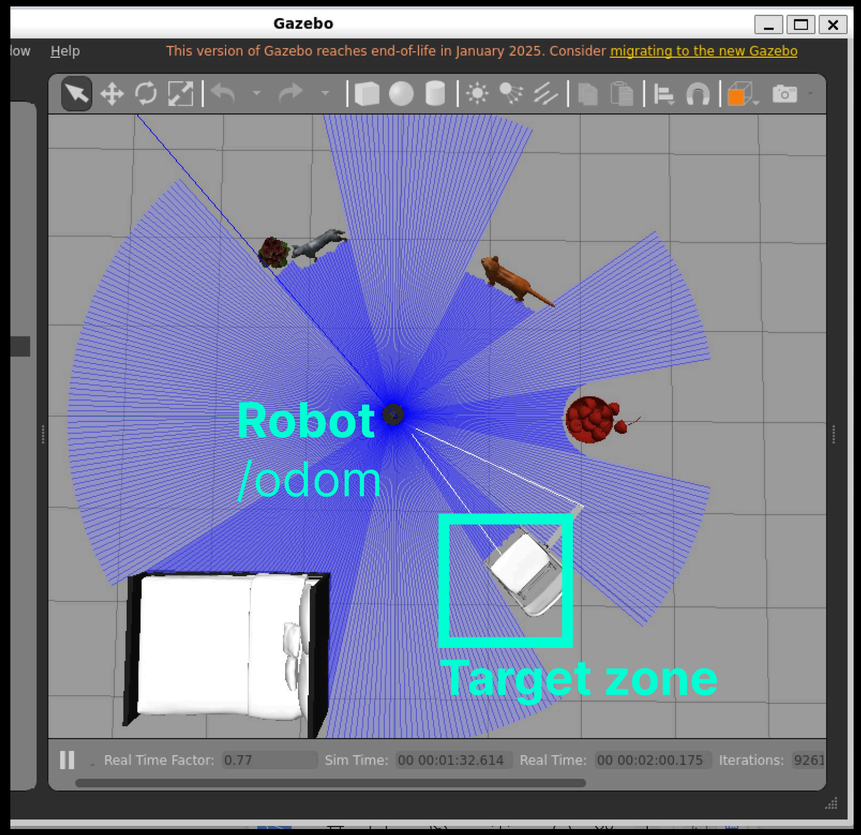


Outputs: vision/raw_detections, /vision/detections, /vision/yolo_debug

Implementation: Odom-Based Navigation

Use /odom and /cmd_vel direct control for a stable demo

- Read current pose (x, y, yaw) from /odom
- Use *_zone targets from config/target.yaml
- Compute target heading and yaw error
- Rotate first if yaw error is large
- Move forward while correcting heading
- Return success within goal tolerance



Implementation: Sequence Executor

Run small action units and manage execution status

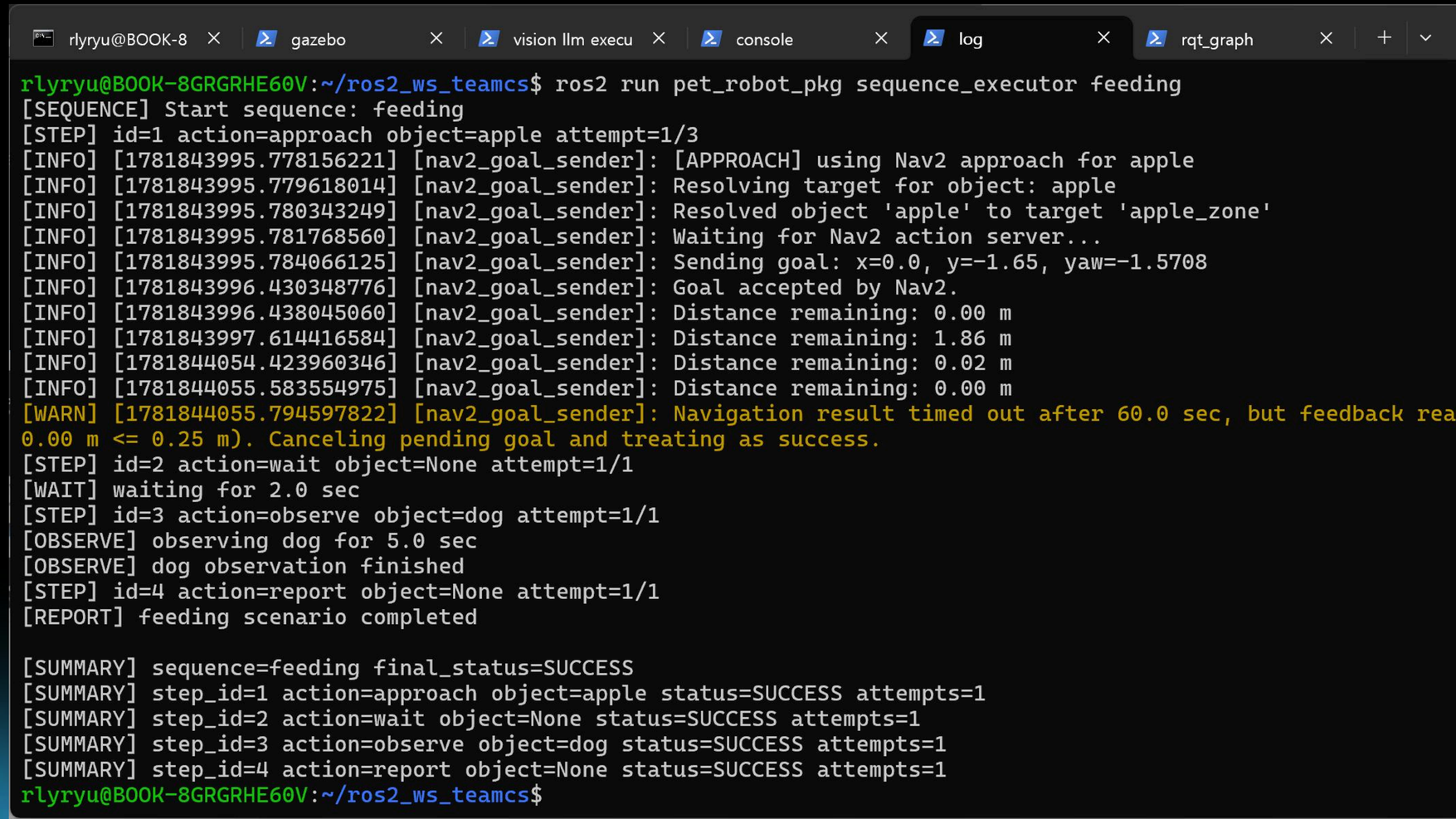
Receive JSON sequence

Validate schema / params

Run step action function

Status success / fail / timeout

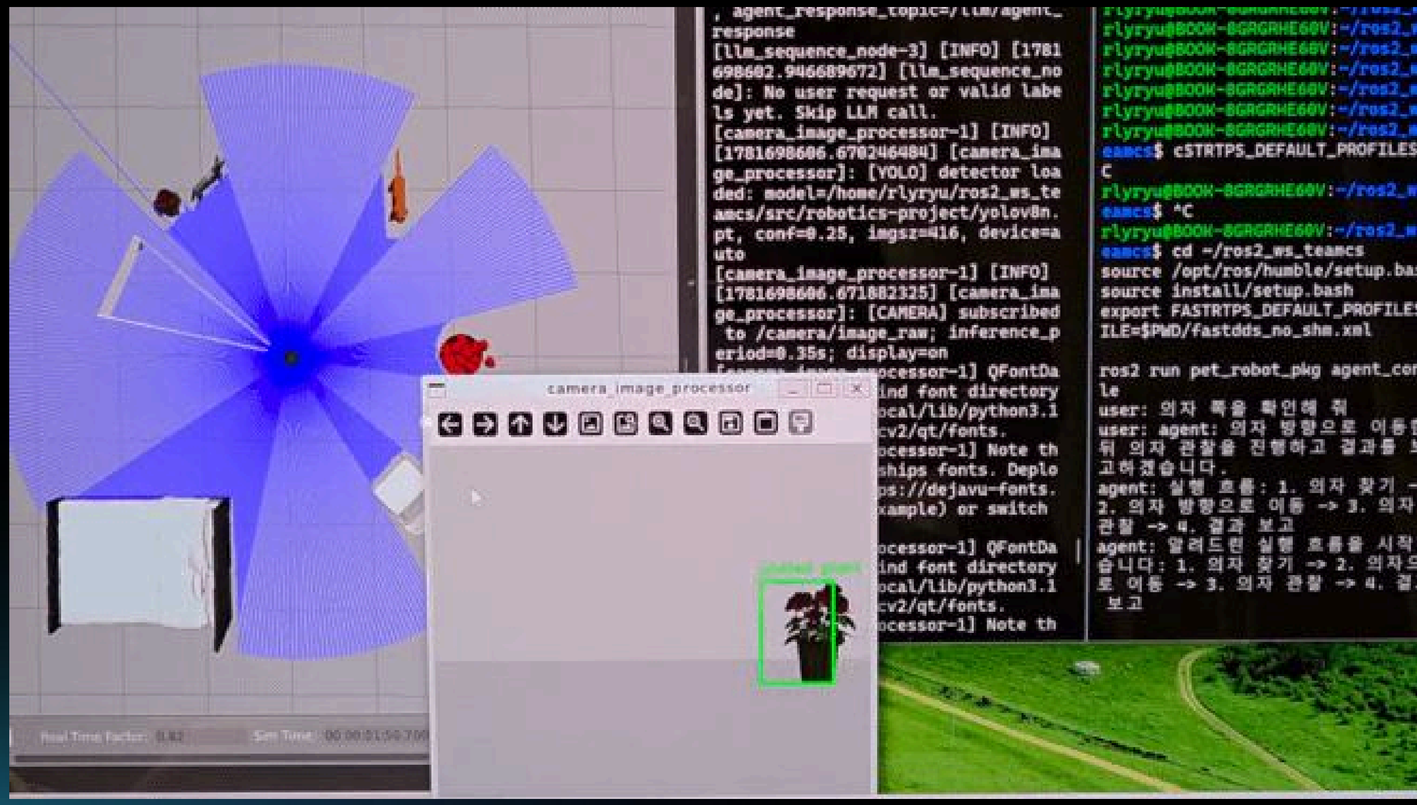
Publish execution status



Results

End-to-end flow from natural language to robot motion

- Runs TurtleBot3, camera, lidar, and odom plugin in Gazebo
- Connects YOLO detection with LLM sequence generation
- Confirms odom-based appoach action behavior
- Supports user input through agent_console



Challenges

Main issues: timing, schema stability, and localization reliability

- **Nav2/map alignment —** AMCL, map pose, and launch dependencies were sensitive
- **LLM output variability —** Validation and fallback handling were required
- **YOLO detection noise —** Frame-level fluctuations needed filtering
- **ROS topic timing —** Node order and process management mattered
- **Target coordinates —** Object centers and safe approach zones had to be separated

Further work

Extend the navigation demo toward robust robot service

Current Demo

LLM + YOLO
+ odom approach

Extension

obstacle avoidance
+ manipulation

Next Step

coordinate integration
+ tracking

Real Robot

TurtleBot deployment
+ metrics

Thank You

Any questions?